

Prof Lin WeiSi
School of Computer Science and Engineering, NTU
Email: wslin@ntu.edu.sg

- This last section on computer arithmetic talks about some considerations to take note during coding.



Effects of four operators

- **Addition/Subtraction**

- Addition/Subtraction will cause the result to increase/decrease
- When done sufficient number of times, **overflow** at Min or Max end of the representable range will eventually occur
- Take note of the range of the **data type** used when coding in High level language, or the **width of registers and memory** when coding in assembly

- **Multiplication**

- Multiplication in binary is similar to an **arithmetic left shift**
- **Overflow** at Min or Max end of the representable range
- Similar considerations as Add/Sub.

- **Division**

- Division in binary is similar to an **arithmetic right shift**
- Truncation of LSB leads to **loss in precision**
- **Reduces the magnitude** of the result so it get further away from the Min/Max of the range.

- First is on the effects of the four basic operations add, subtract, multiply and divide, specifically dealing with overflow and accuracy of result.
- Addition and subtraction, if done multiple times, may potentially lead to result overflowing beyond one end of the range.
 - When that occurs, the result obtained will be wrong.
 - One example is when the result is larger than what a particular data type could hold, such as storing a value larger than 255 to the unsigned character data type,
 - Or trying to store a value larger than 255 to a 8-bit register.
- Similarly, multiplication will result in a larger number if the multiplier is larger than one, which means potentially may lead to overflow condition.
- Division with a divisor larger than one, on the other hand, gave raise to a different issue, which is the truncation of the lower order bits or digits.
 - This is easily illustrated by the fact division is similar to arithmetic right shift.
 - Any truncation of lower order bits means a loss in accuracy of the data.
- So take note of the effects of these operators so as to have the best accuracy without running into overflow condition.



Effects of Rounding

- Rounding refers to **removing of LSB(s)** so that the result can fit into the representable bits.
 - The limits imposed in the width of the representable bits could be from the **registers width, data type etc.**
- Rounding can be round-up or round-down to nearest representable number.
 - As the rounded number is an approximation of the raw result, a certain amount of **rounding error is incurred.**
- Below an example of rounding off a floating point number to 2 decimal points, incurring an **error of 0.004×10^1**

1 • 6 8 6 e + 0 1 $\longrightarrow$ 1 • 6 9 e + 0 1

- Common to have **intermediate register with width larger than regular data registers** to allow intermediate processing to be done at **higher precision** and thus **reducing the amount of rounding error.**

- Next is the rounding process.
- Rounding is a process in which the lower order bits of a data is truncated in order to fit the data into a register or memory location of a smaller data width.
 - Some approximation needs to be done when performing the rounding and this is by rounding up, or down to the nearest representable number.
- Since the rounded number is just an approximation of the actual data, a certain amount of rounding error is incurred in the process.
 - For example, rounding 1.686×10^01 to 1.69×10^01 result in rounding error of 0.004×10^01 .
- It is common to have intermediate registers that have larger data width compared to the regular data registers in a processor. This is to preserve the intermediate data without truncating the lower order bits. Reducing the rounding error.

Handling numbers with different magnitudes

Example:

$$\begin{aligned} &1.00 \times 10^0 + 1.00 \times 10^0 + \\ &1.00 \times 10^0 + 1.00 \times 10^0 + \\ &1.00 \times 10^0 + 1.23 \times 10^3 \end{aligned}$$

- The **order of evaluation** can affect accuracy of result
 - Add/subtract operands with **similar size of magnitude** first.

	1	•	0	0	0	e	+	0	0
+	1	•	0	0	0	e	+	0	0
	2	•	0	0	0	e	+	0	0
+	1	•	0	0	0	e	+	0	0
	3	•	0	0	0	e	+	0	0
+	1	•	0	0	0	e	+	0	0
	4	•	0	0	0	e	+	0	0
+	1	•	0	0	0	e	+	0	0
	0	•	0	0	5	e	+	0	3
+	1	•	2	3	0	e	+	0	3
	1	•	2	4	0	e	+	0	3

Align

Rounding

- A different result would be obtained if we change the sequence of accumulation.
- If we allow the small number to accumulate first before doing any truncation, it will allow these small numbers to accumulate to a significant value and not be truncated off during rounding process.
- In this example, a raw intermediate value 1.235×10^3 is obtained.
- After rounding, we get 1.24×10^3 , i.e. the significance of the '1's are not truncated as in the case of the previous slide.
- In general, when performing arithmetic operation between numbers with huge difference in magnitude, always add/subtract between numbers that are of similar magnitude before combining them to other larger numbers.

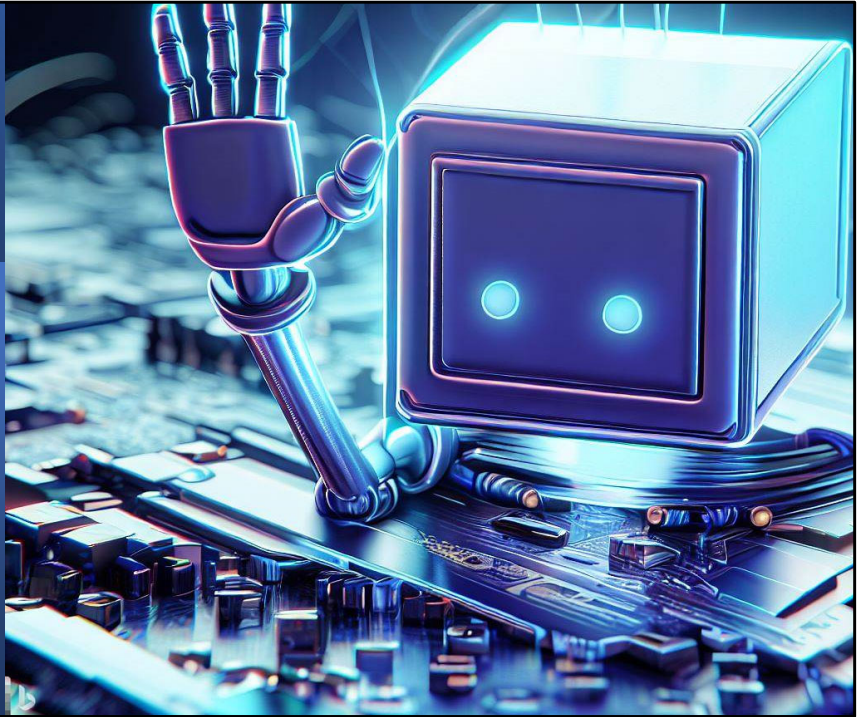


Maximising Accuracy during computation

- Two issues: **overflow and precision**.
- From previous slides, **addition, subtraction and multiplication** may potentially lead to **result overflowing** the range of the representable number used.
- **Division** will lead to **loss in precision** as bits are lost due to truncation of LSB(s).
- **Some rule of thumb**
 - **Accumulate/subtract** numbers with **small magnitude first** to allow their magnitude to be comparable to big magnitude numbers.
 - Take note of the range of the number system used, which, depending on the usage scenario, can be a factor of the **data type, number format, register/memory width** etc.
 - **check for overflow** in a process if possible.
 - To **preserve** as much **precision** as possible, always **do division last** as far as possible.

- To summarise what we have discussed in this section of overflow and accuracy.
- Some general rule of thumb that we can use when designing algorithm
 - Accumulate numbers with small magnitude first before merging them to calculation involving large numbers so that they would not be truncated off prematurely.
 - Take note of the range of the various data type, number format, register, memory etc
 - if an overflow occurs during computation, that automatically implies that the result obtained is wrong.
 - Try to perform division operation last to preserve as much precision as possible for the intermediate data.
- This is the last slide for the computer arithmetic chapter.

Thank you



No part of this material shall be filmed, recorded, downloaded, reproduced, distributed, republished, or transmitted in any form or by any means without written approval from Nanyang Technological University.